Introduction to Deep Learning

MODULE 1

Deep Learning vs Traditional Machine Learning

Conceptual Foundation

Traditional Machine Learning (ML): Traditional ML focuses on learning a mapping $f:X \rightarrow Y$ using **explicit feature representations** engineered by domain experts. Model performance is heavily dependent on the quality of feature extraction and selection.

Deep Learning (DL): Deep Learning models learn hierarchical representations of data using multi-layer neural networks, enabling end-to-end learning from raw input to output without explicit feature engineering.

Feature Representation

Aspect	Traditional ML	Deep Learning
Feature design	Handcrafted, domain-specific	Automatically learned
Representation	Shallow, fixed	Hierarchical, distributed
Feature optimization	Separate from model training	Jointly optimized via backpropagation

Model Architecture

Aspect	Traditional ML	Deep Learning
Depth	Shallow (1–2 layers)	Deep (tens to hundreds of layers)
Non-linearity	Limited	Highly expressive
Learning paradigm	Often modular	End-to-end learning

Data and Sample Complexity

Aspect	Traditional ML	Deep Learning
Sample efficiency	High	Low
Data dependency	Performs well with limited data	Requires large-scale datasets
Generalization	Relies on regularization & feature design	Achieved via representation learning

Optimization and Training

Aspect	Traditional ML	Deep Learning
Optimization	Convex or quasi-convex	Highly non-convex
Training stability	More stable	Sensitive to initialization & hyperparameters
Learning algorithm	Often closed-form or simple gradients	Backpropagation with SGD variants

Computational Requirements

Aspect	Traditional ML	Deep Learning
Hardware	CPU-based	GPU/TPU essential
Training time	Short	Long
Memory	Low	Very high

Interpretability and Theoretical Guarantees

Aspect	Traditional ML	Deep Learning
Interpretability	High (e.g., linear models, trees)	Low (black-box models)
Theoretical guarantees	Strong (VC theory, convex optimization)	Limited, still evolving
Debuggability	Easier	Difficult

Performance Across Data Types

Data Type	Traditional ML	Deep Learning
Structured/ tabular	Often superior	Not always optimal
Images, audio, text	Weak	State-of-the-art
Sequential data	Feature-depen dent	Naturally handled by RNNs/Transformers

Bias–Variance Tradeoff

- Traditional ML: Bias controlled via feature engineering and model selection
- Deep Learning: Low bias, high variance; mitigated using large data, dropout, batch normalization

Mathematical Perspective

- Traditional ML optimizes a fixed feature space
- Deep Learning jointly optimizes:

Representation + Classifier

making it more expressive but harder to analyze theoretically.

Typical Algorithms

Traditional ML

- Linear/Logistic Regression
- SVM
- Decision Trees, Random Forest
- KNN, Naïve Bayes

Deep Learning

- CNN (Computer Vision)
- RNN / LSTM / GRU (Sequences)
- Transformers (NLP, Vision, Multimodal)

Gradient Descent

Gradient descent, how neural networks learn | Deep Learning Chapter 2

Introduction

- Gradient Descent is an iterative optimization algorithm used to minimize the loss (cost) function of a model by updating parameters in the direction of steepest descent.
- In deep learning, it is the core mechanism behind training neural networks.

Objective

- Given:
  - Model parameters: θ
  - Loss function: J(θ)
- Goal:

$$\theta^* = \arg\min_{\theta} J(\theta)$$

  - Gradient Descent moves θ towards the minimum of J(θ).

Mathematical Foundation

- The gradient of a function is a vector of partial derivatives:

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \\ \vdots \end{bmatrix}$$

- It points in the direction of maximum increase of the function.

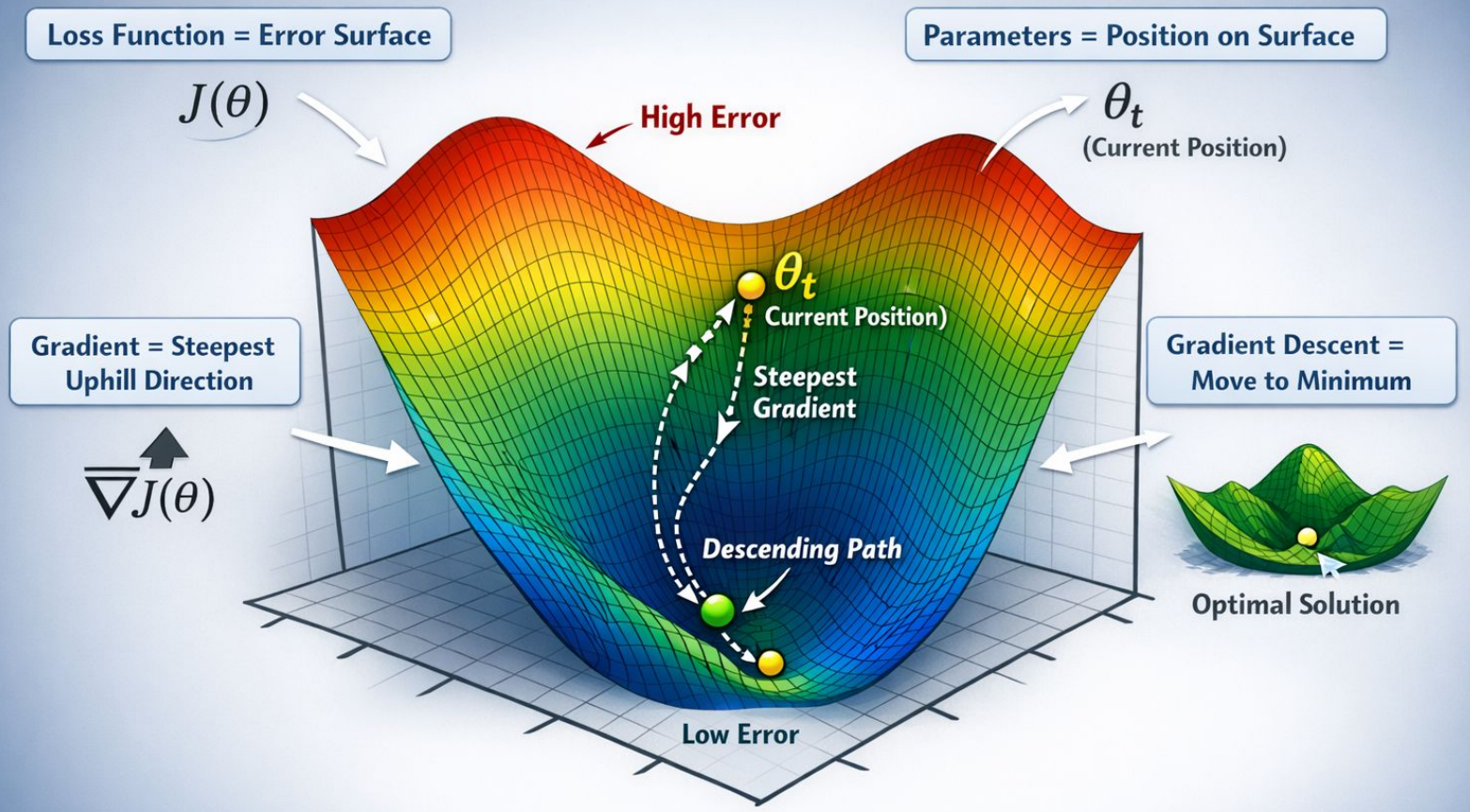
- Update Rule

$$\theta^{(t+1)}=\theta^{(t)}-\eta\nabla J(\theta^{(t)})$$

Where:

- η (learning rate) controls step size
- Minus sign $\longrightarrow$ move opposite to gradient (downhill)

Intuition of Gradient Descent in Deep Learning

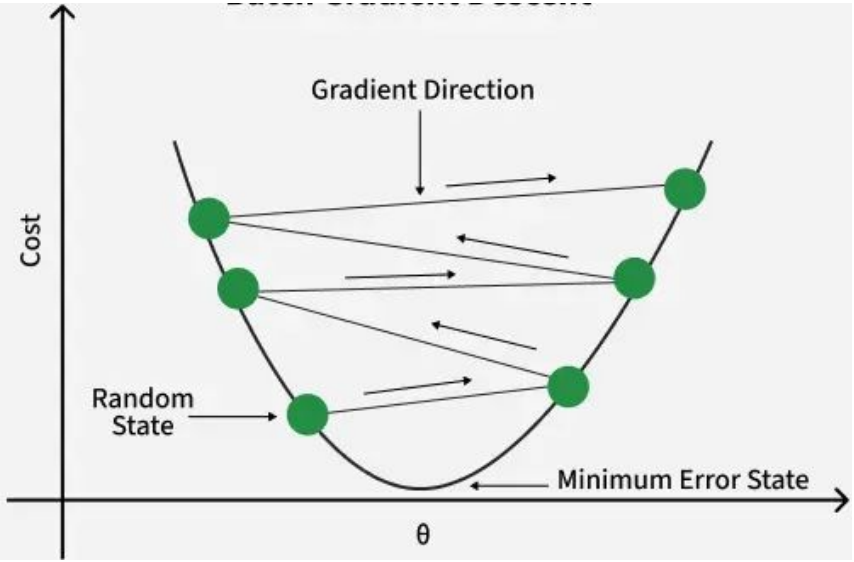


Types of Gradient Descent

- Batch Gradient Descent: In each iteration the algorithm calculates the average gradient of the loss function for all the training examples and updates the model parameters accordingly.
  - Uses entire training dataset for each update.

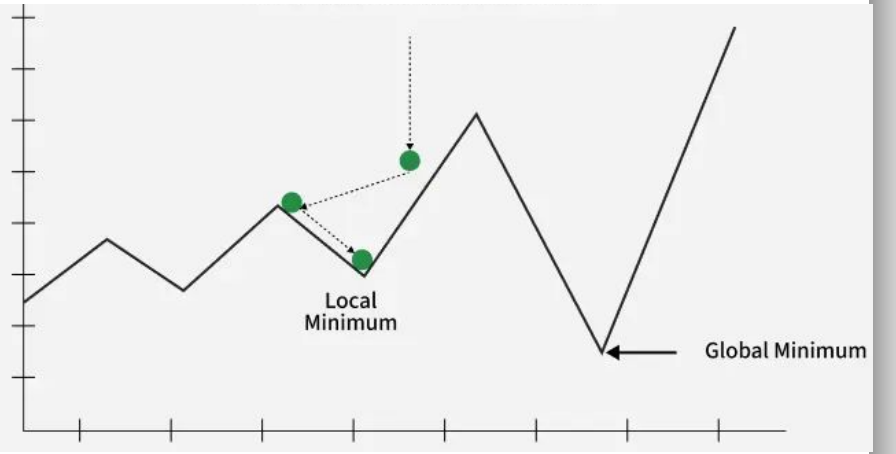
$$\theta = \theta - \eta \frac{1}{n} \sum_{i=1}^{n} \nabla J_i(\theta)$$

- Pros
  - Stable convergence
  - Accurate gradient
- Cons
  - Slow
  - Memory intensive



- <u>Stochastic Gradient Descent (SGD)</u>
  - Uses one training sample at a time. Updates the parameters more frequently, leading to faster convergence.

$$\theta = \theta - \eta \nabla J_i(\theta)$$

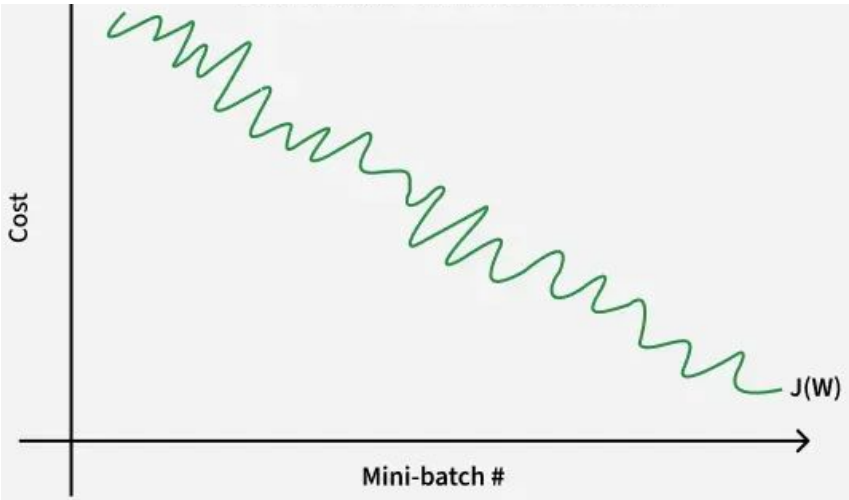


- Pros
  - Faster
  - Escapes local minima
- Cons
  - Noisy updates
  - Oscillations near minimum

- <u>Mini-Batch Gradient Descent (Most Used)</u>
  - In between Batch GD and Stochastic GD. Uses small batch (e.g., 32, 64).

$$\theta = \theta - \eta \frac{1}{b} \sum_{i=1}^{b} \nabla J_i(\theta)$$

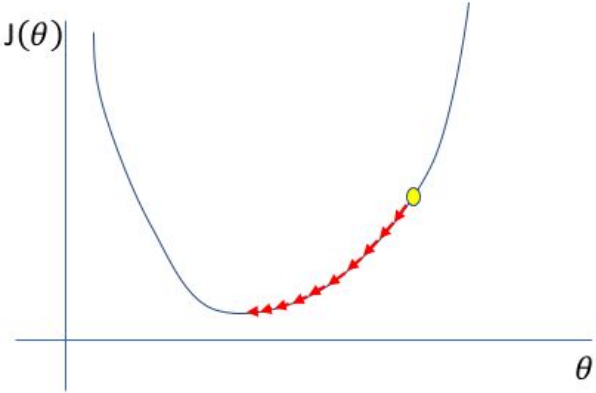
- Pros
  - Fast
  - Stable
  - GPU-friendly



Learning Rate (η)

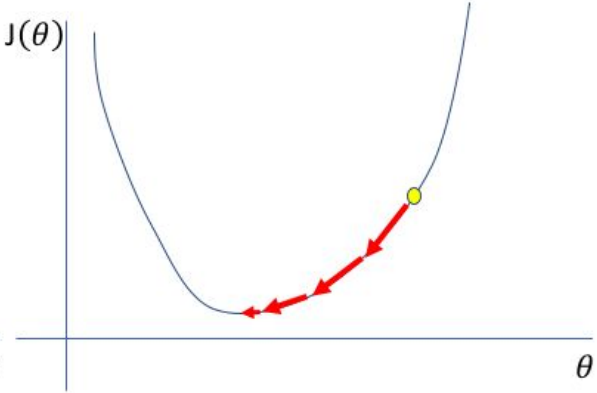
- Importance
  - Too small $\longrightarrow$ very slow convergence
  - Too large $\longrightarrow$ divergence or overshooting
- Techniques
  - Learning rate decay
  - Adaptive learning rates

Too low



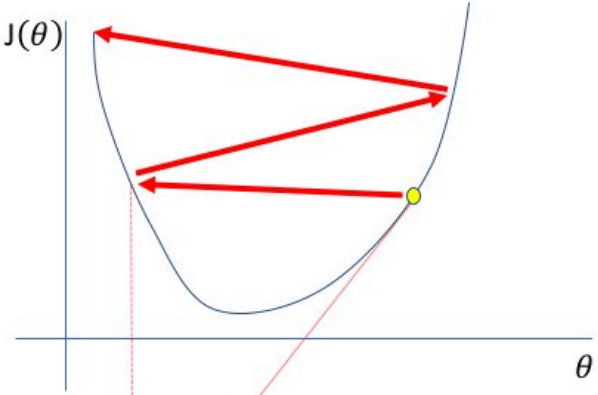
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high



Too large of a learning rate causes drastic updates which lead to divergent behaviors

Gradient Descent in Neural Networks

- Training Steps
  1. Forward propagation
  2. Compute loss
  3. Backpropagation (chain rule)
  4. Gradient computation
  5. Parameter update using Gradient Descent

Convergence Criteria

- Gradient Descent stops when:
  - Gradient ≈ 0
  - Loss change < threshold
  - Max epochs reached

Gradient Descent Variants - 1. Momentum

- Enhancement of standard gradient descent.
- Aims to accelerate convergence particularly in the presence of high curvature, small but consistent gradients or noisy gradients.
- Introduces a velocity term that accumulates the gradient of the loss function over time thereby smoothing the path taken by the parameters.

$$v_t = \gamma v_{t-1} + \eta \nabla J(\theta)$$

$$\theta = \theta - v_t$$

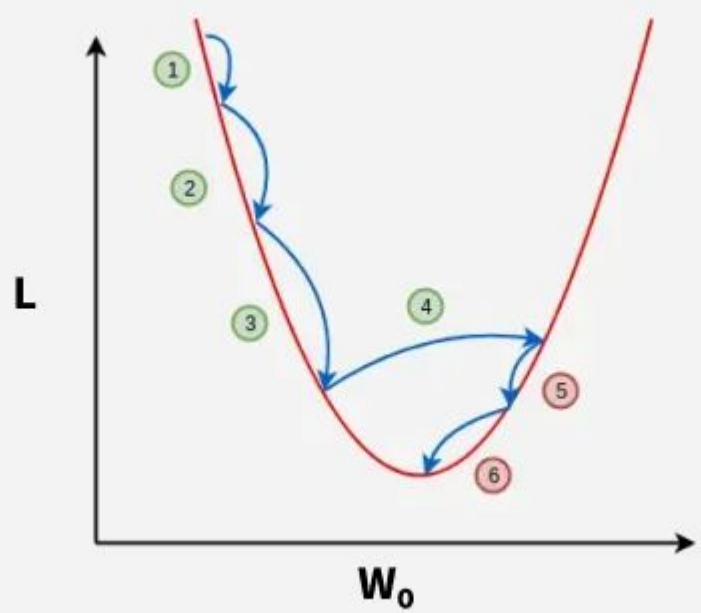
v_t is the velocity at iteration t,

γ is the momentum term (typically between 0 and 1),

η is the learning rate,

$\nabla J(\theta_t)$ is the gradient of the loss function

$J(\theta)$ with respect to θ at iteration t.



Advantages

- **Accelerated Convergence:** Helps in faster convergence especially in scenarios with small but consistent gradients.
- **Smoother Updates:** Reduces the oscillations in the gradient updates, leading to a smoother and more stable convergence path.
- **Effective in Ravines:** Particularly effective in dealing with ravines or regions of steep curvature and is common in deep learning loss landscapes.

Disadvantages

- **Additional Hyperparameter:** Introduces an additional hyperparameter (momentum term) that needs to be tuned.
- **Complex Implementation:** Slightly more complex to implement compared to standard gradient descent.
- **Potential Overcorrection:** If not properly tuned the momentum can lead to overcorrection and instability in the updates.

2. Adagrad (Adaptive Gradient Algorithm)

- Adjusts the learning rate for each parameter during training.
- Unlike standard gradient descent with a fixed rate, Adagrad uses past gradients to scale updates making it effective for sparse data and varying feature magnitudes.

Working

- The learning rate is adapted based on the historical sum of squared gradients for each parameter.

  1. Initialization: Initialize the parameter values randomly, just like other optimization algorithms. Additionally, it initializes a running sum of squared gradients for each parameter which will track the gradients over time.

  2. Gradient Calculation: For each training step, the gradient of the loss function with respect to the model's parameters is calculated, just like in standard gradient descent.

  3. Adaptive Learning Rate: Adagrad adjusts the learning rate for each parameter based on the accumulated sum of squared gradients.

The updated learning rate for each parameter is calculated as

$$lr_t = \frac{\eta}{\sqrt{G_t + \epsilon}}$$

Where:

η is the global learning rate (a small constant value)

G_t is the sum of squared gradients for a given parameter up to time step t

ϵ is a small value added to avoid division by zero (often set to 1e−8)

Here, the denominator $G_t+\epsilon$ grows as the squared gradients accumulate, causing the learning rate to decrease over time which helps to stabilize the training.

4. Parameter Update: The model's parameters are updated by subtracting the product of the adaptive learning rate and the gradient at each step:

$$\theta_{t+1} = \theta_t - lr_t \cdot \nabla_\theta J(\theta)$$

Where:

θ_t is the current parameter

$\nabla_\theta J(\theta)$ is the gradient of the loss function with respect to the parameter

- Adagrad is ideal for:
  - Problems with sparse data and features like in natural language processing or recommender systems.
  - Tasks where features have different levels of importance and frequency.
  - Training models that do not require a very fast convergence rate but benefit from a more stable optimization process.
  - However, if you are dealing with problems where a more constant learning rate is preferable, using variants like RMSProp or Adam might be more appropriate.

Advantages

- Adapts learning rates for each parameter, helping with sparse features and noisy data.
- Works well with sparse data by giving rare but important features appropriate updates.
- Automatically adjusts learning rates, eliminating the need for manual tuning.
- Improves performance in cases with varying gradient magnitudes, enabling efficient convergence.

Disadvantages

- Learning rates shrink continuously during training which can slow convergence and cause early stopping.
- Performance depends heavily on the initial learning rate choice.
- Lacks momentum, making it harder to escape shallow local minima.
- Learning rates decrease as gradients accumulate which helps avoid overshooting but may hinder progress later in training.

3. RMSProp

- RMSProp (Root Mean Square Propagation) is an adaptive learning rate optimization algorithm designed to improve the performance and speed of training deep learning models.
- This variant adapts the learning rate for each parameter individually by considering the magnitude of recent gradients for those parameters.
- Helps in dealing with the challenges of non-stationary objectives and sparse gradients commonly encountered in deep learning tasks.

Need for RMSProp

- Developed to overcome limitations of earlier optimization methods such as SGD and AdaGrad.
  - SGD (Stochastic Gradient Descent) uses a constant learning rate, which can be inefficient during training.
  - AdaGrad reduces the learning rate too aggressively, causing slow learning at later stages.
- RMSProp addresses these issues by adapting learning rates using a moving average of squared gradients.
- This adaptive mechanism maintains a balance between efficient convergence and training stability.
- As a result, RMSProp is widely used in modern deep learning applications

Mathematical Formulation

1. Compute the gradient g_t at time step t: $g_t = \nabla \theta$
2. Update the moving average of squared gradients:

   $E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma)$ where γ is the decay rate.
3. Update the parameter θ using the adjusted learning rate:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_{t-1} + \epsilon}}$$

where η is the learning rate and ϵ is a small constant added for numerical stability.

Parameters Used in RMSProp

- Learning Rate (η): Controls the step size during the parameter updates. RMSProp typically uses a default learning rate of 0.001, but it can be adjusted based on the specific problem.
- Decay Rate (γ): Determines how quickly the moving average of squared gradients decays. A common default value is 0.9 which balances the contribution of recent and past gradients.
- Epsilon (ϵ): A small constant added to the denominator to prevent division by zero and ensure numerical stability. A typical value for ϵ is 1e-8.

By carefully adjusting these parameters, RMSProp effectively adapts the learning rates during training, leading to faster and more reliable convergence in deep learning models.

Advantages

- Adaptive Learning Rates: Adjusts learning rates for each parameter individually, optimizing updates more effectively.
- Handles Non-Stationary Objectives: Efficiently adapts to changing optimal parameter values over time.
- Prevents Learning Rate Decay Problem: Maintains optimal learning rates by using a decay rate unlike AdaGrad.
- Improved Convergence Speed: Faster convergence due to balanced and dynamic learning rates.

Disadvantages

- Sensitivity to Hyperparameters: Performance is sensitive to settings like decay rate and epsilon meaning it requires careful tuning.
- Poor Performance with Sparse Data: May struggle with sparse data, leading to slower or inconsistent convergence.

4. Adam (Adaptive Moment Estimation)

- Adam combines the benefits of AdaGrad and RMSProp to achieve effective and adaptive learning rates.
- It maintains two moving average variables:
  - v - For the first moment
  - s - For the second moment
- The algorithm computes an exponentially weighted average of the past gradients and their squared gradients. These moving averages are then used to update the model's parameters.

Working

1. Initialize the Variables: Initialize the moving average variables v and s as dictionaries to store the exponentially weighted averages of the gradients and squared gradients, respectively.
2. Compute Moving Averages: For each parameter of the model, compute the moving average of the gradients by combining the current gradient with the previous moving average. It also computes the moving average of the squared gradients.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

m_t: first moment estimate (mean of gradients)
g_t: gradient at time step t
β_1: decay rate (typically 0.9)

$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$

v_t: second moment estimate (variance of gradients)

β_2: decay rate (typically 0.999)

3. Bias Correction: To reduce bias during the initial iterations, bias correction is done by dividing the moving averages by a correction factor.

   Because moving averages are biased toward zero initially:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

4. Update Parameters: Use the moving averages of the gradients and squared gradients to update each parameter.

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Method	Speed	Stability	Usage
Batch GD	Slow	High	Small datasets
SGD	Fast	Low	Online learning
Mini-Batch GD	Fast	High	Deep learning
Adam	Very Fast	Very High	Default choice

Neural Network Optimisation

- Challenges
  - Neural networks have millions of parameters
  - Loss surface is:
    - Non-convex
    - High dimensional
    - Highly irregular
  - Optimization difficulties directly affect:
    - Convergence speed
    - Training stability
    - Final accuracy

What is Ill-Conditioning?

- A problem is ill-conditioned when:
  - The loss surface has very different curvatures in different directions
  - Small parameter changes $\longrightarrow$ large loss changes in some directions, small in others

Mathematical View

- Consider Hessian matrix H of the loss function:

  $H = \nabla^2 J(\theta)$

- Condition number: $\kappa(H) = \frac{\lambda_{max}}{\lambda_{min}}$

  Large $\kappa \longrightarrow$ ill-conditioned problem

- Loss contours are elongated ellipses
- Gradient descent:
  - Oscillates across steep direction
  - Slowly progresses along flat direction

- Loss contours are elongated ellipses
- Gradient descent:
  - Oscillates across steep direction
  - Slowly progresses along flat direction
- It causes
  - Very slow convergence
  - Zig-zag trajectory
  - Sensitive to learning rate

Causes of ill-condition in NN

- Poor weight initialization
- Highly correlated input features
- Deep architectures
- Poorly scaled data

Causes of ill-condition in NN

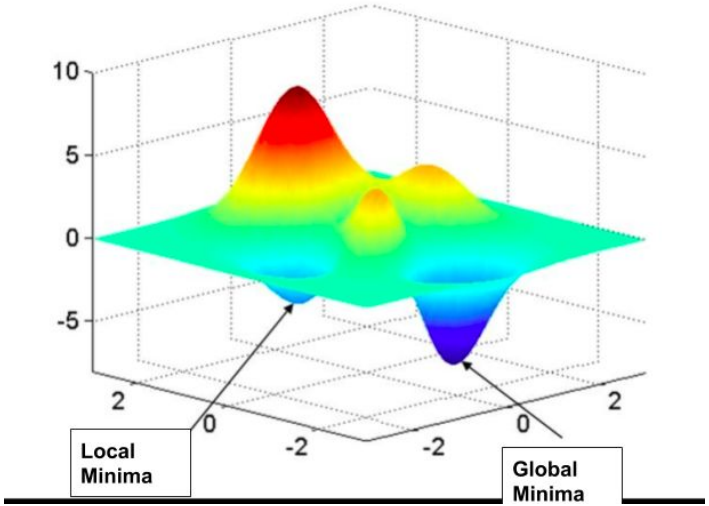
- Poor weight initialization
- Highly correlated input features
- Deep architectures
- Poorly scaled data
- Solutions
  - Feature normalization
  - Batch Normalization
  - Adaptive optimizers (Adam, RMSProp)
  - Second-order methods (limited use)

Challenges in Gradient Descent

- Local Minima
  - Algorithm may get stuck in shallow minima.
- Saddle Points
  - Flat regions with zero gradient slow convergence.
- Vanishing Gradient
  - Gradients become very small (common in deep networks).
- Exploding Gradient
  - Gradients grow exponentially.

Local Minima

- A local minimum is a point where:
  - $J(\theta) \leq J(\theta + \epsilon)$
- for small ε, but not the global minimum



- In high-dimensional spaces:
  - Poor local minima are rare
  - Most local minima are good enough
- This is why deep networks work well in practice.

Concerns

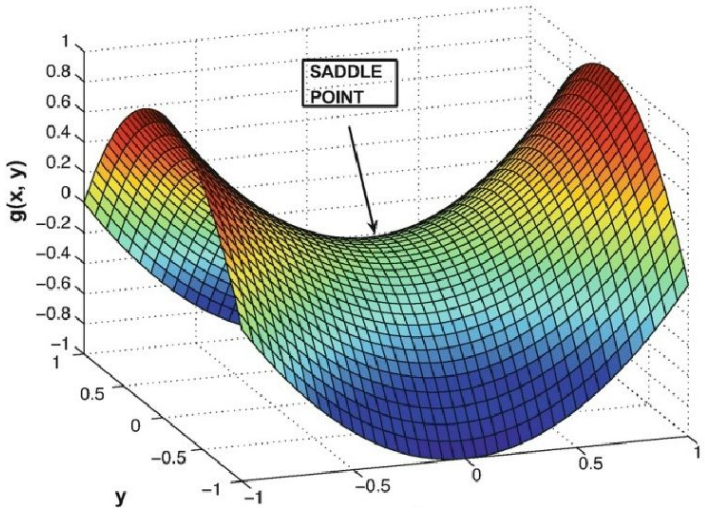
- Shallow local minima can slow convergence: Gradients are very small in flat regions, so parameter updates become slow. Loss value is lower than nearby points. The surface around it is flat or gently curved. Small parameter changes do not significantly increase the loss.
- Small datasets $\longrightarrow$ higher risk of poor local solutions
- Bad initialization increases chances of getting stuck in local minima or saddle points

How SGD helps

- Noise in SGD helps escape shallow minima
- Mini-batch randomness acts as exploration
- Solution to local minima
  - Random initialization
  - SGD with momentum
  - Learning rate schedules
  - Ensemble training

Saddle Points

- A saddle point is:
  - Minimum in one direction
  - Maximum in another direction
- Mathematically:
  - $\nabla J(\theta)=0$
- but Hessian has both positive and negative eigenvalues



- Why Saddle Points are Dangerous
  - Gradient $\approx 0 \longrightarrow$ training stalls
  - Much more common than local minima
  - Increase exponentially with dimensions
- Example
  - Think of a mountain pass:
    - Flat region
    - No clear downhill direction

- Why Deep Networks Suffer
  - Millions of parameters
  - Huge number of saddle points
  - Flat plateaus around saddle points
- Solutions
  - Momentum-based methods
  - Adam optimizer
  - Noise injection
  - Second-order curvature information

Exploding Gradients

- Gradients grow exponentially large during backpropagation:
  - $// \nabla J(\theta) // \rightarrow \infty$
- Where It Happens
  - Deep networks
  - Recurrent Neural Networks (RNNs)
  - Long sequences
  - Large weights
- Mathematical Cause
  - Backpropagation involves repeated multiplication: $\prod_{i=1}^{L} w_i$
  - If eigenvalues > 1 $\rightarrow$ explosion

- Effect
  - Unstable training
  - Loss becomes NaN
  - Weights diverge
- Solutions
  - Gradient clipping
  - Proper weight initialization (Xavier, He)
  - Batch normalization
  - Smaller learning rates
  - Use LSTM/GRU for RNNs

Challenge	Main Issue	Common Solution
Ill-conditioning	Slow convergence	BatchNorm, Adam
Local minima	Suboptimal solution	SGD, initialization
Saddle points	Training stagnation	Momentum, Adam
Exploding gradients	Instability	Gradient clipping

Weight Initialization Strategies

- Weight initialization is the pre-training step of assigning initial values to a neural network's trainable parameters (weights and biases).
- Proper initialization is critical because it dictates the starting point on the non-convex loss landscape.
- Directly impacting convergence speed and the quality of the final model.

Key Challenges in Initialization

- Symmetry Breaking: If weights are initialized to the same constant (like zero), all neurons in a layer will perform identical computations and receive identical gradient updates, effectively acting as a single neuron.
- Vanishing Gradients: Initializing with weights that are too small causes the signal to shrink as it passes through layers, making gradients so tiny that the network stops learning.
- Exploding Gradients: Initializing with weights that are too large causes the signal to grow exponentially, leading to numerical instability and divergent training.

Zero Initialization

- All the weights are assigned zero as the initial value in zero initialization.
- Highly ineffective as neurons learn the same feature during each iteration.
- Any kind of constant initialization, the same issue happens to occur.
- The outputs of all hidden units in the model would have the same influence on the cost, thus, resulting in identical gradients.
- Thus, constant initializations are not preferred.

Random Initialization

- Random initialization assigns random values except for zeros as weights to neuron paths to break symmetry.
- Problems like Overfitting, Vanishing Gradient Problem, Exploding Gradient Problem may occur.
- Random Initialization can be of two kinds:
  - Random Normal: The weights are initialized from values in a normal distribution. Bell shaped - Central values has more probability of occurrence.
  - Random Uniform: The weights are initialized from values in a uniform distribution. Rectangular - All values have equal probability of occurrence.

Xavier/Glorot Initialization

- Solves the problem of vanishing and exploding gradients
- It aims to address the issue of maintaining variance when using certain activation functions like tanh and the logistic sigmoid.
- Uniform Xavier Initialization:
  - Weights are initialized by drawing them from a random uniform distribution within a specific range determined by the formula:

$$x = \sqrt{\frac{6}{n_{inputs} + n_{outputs}}}$$

    - n - No. of inputs/outputs

- For each weight in network we draw a random value w from a uniform distribution in the range [-i, i].
- Constraining the weights within a range determined by x ensures that the variance of the initial weights is controlled and is suitable for effective training.

- Normal Xavier Initialization
  - Sets the initial weights by drawing from a gaussian distribution with a mean of 0 and a standard deviation σ, which is determined by the formula:

$$\sigma = \sqrt{\frac{2}{n_{inputs} + n_{outputs}}}$$

  - it adjusts the scale of the weights in a way that keeps the network's activations within a reasonable range, regardless of the layer size.

Kaiming Initialization (He Initialization)

- Addresses vanishing and exploding gradient problems when using ReLU activation function.
- Calculated as a random number with a Gaussian probability distribution (G) with a mean of 0.0 and a standard deviation of sqrt(2/n), where n is the number of inputs to the node.
  - $W \longrightarrow N(0, \sqrt{\frac{2}{n_{inputs}}})$
  - The factor of 2 in the variance is specific to the ReLU activation function.

Batch Normalization

Internal Covariate Shift

- In traditional neural networks as the input data propagates through the network, the distribution of each layer's inputs changes.
- This phenomenon is known as internal covariate shift and it can slow down training process.
- Batch Normalization aims to reduce this issue by normalizing the inputs of each layer.
- This process keeps the inputs to each layer of the network in a stable range even if the outputs of earlier layers change during training. As a result training becomes faster and more stable.

- Batch Normalization is used to reduce the problem of internal covariate shift in neural networks.
- A batch normalization layer looks at each batch as it comes in.
- Each batch is first normalized with its own mean and standard deviation
- Then the data is rescaled with two trainable scaling parameters.

This means it calculates the mean and variance of data in a batch and then adjusts the values so that they have similar range. After that it scales and shifts the values so that model learn effectively.

Need for Batch Normalization

- Batch Normalization makes sure outputs of each layer stay steady as model learns. This helps model train faster and learn more effectively.
    - Solves the problem of internal covariate shift.
    - Makes training faster and more stable.
    - Allows use of higher learning rates.
    - Helps avoid vanishing or exploding gradients.
    - Can act like a regularizer sometimes reduce the need for dropout.

Working

1. Compute the Mean and Variance of each batch.

For mini-batch of activations $x_1, x_2, \ldots, x_m$, the mean μ_B and variance σ_B^2 of the mini-batch are computed.

2. Normalization

Each activation x_i is normalized using the computed mean and variance of the mini-batch by subtracting the mean μ_B from each activation and divides by the square root of the variance σ_B^2, ensuring that the normalized activations have a zero mean and unit variance. Additionally a small constant ϵ is added to the denominator for numerical stability, particularly to prevent division by zero.

$$\hat{x} = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

3. Scale and Shift the Normalized Activations

The normalized activations x_i are then scaled by a learnable parameter γ and shifted by another learnable parameter β. These parameters allow the model to learn the optimal scaling and shifting of the normalized activations giving the network additional flexibility.

$$y_i = \gamma \hat{x}_i + \beta$$

Benefits of Batch Normalization

- Faster Convergence: Batch Normalization reduces internal covariate shift, allowing for faster convergence during training.
- Higher Learning Rates: With Batch Normalization, higher learning rates can be used without the risk of divergence.
- Regularization Effect: Batch Normalization introduces a slight regularization effect that reduces the need for adding regularization techniques like dropout.

By normalizing the inputs to each layer Batch Normalization helps stabilize the learning process and allows for faster convergence, making training more effective and reducing the need for careful initialization and learning rate tuning.

Regularization

Introduction

- A universal problem in machine learning has been making an algorithm that performs equally well on training data and any new samples or test dataset.
- Techniques used in machine learning that have specifically been designed to cater to reducing test error, mostly at the expense of increased training error, are globally known as regularization.

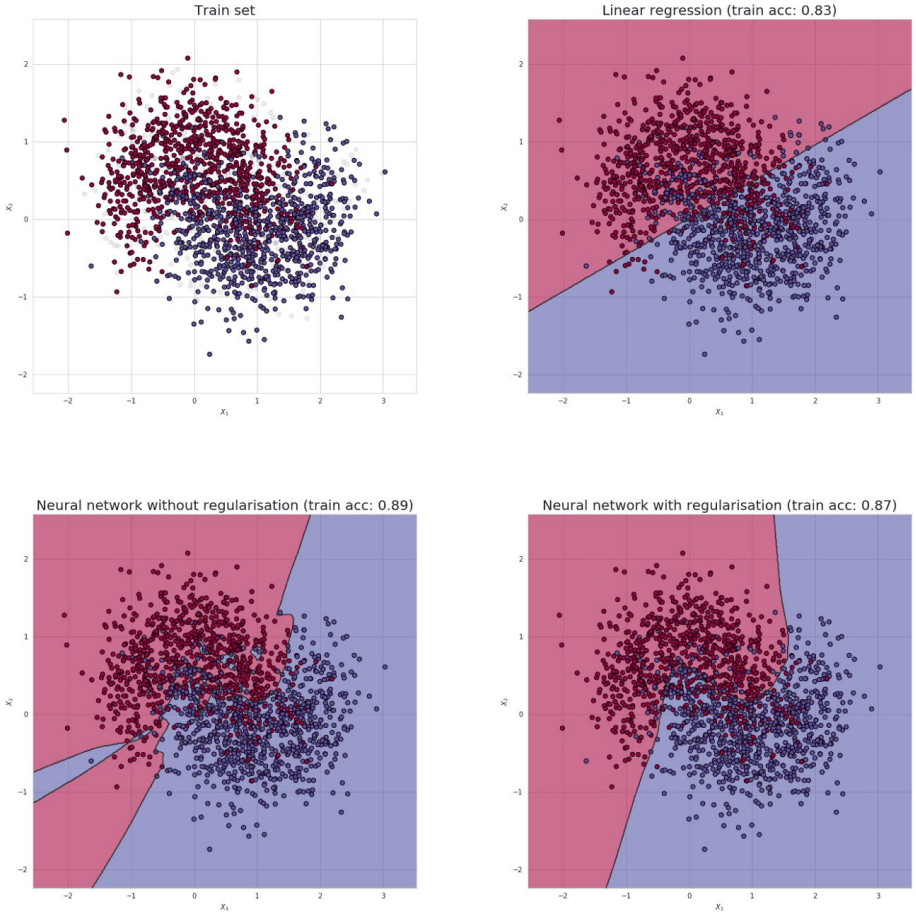
Regularization

- Any modification or change in the learning algorithm that helps reduce its error over a test dataset, commonly known as generalization error but not on the supplied or training dataset is regularization.
- In learning algorithms, there are many variants of regularization techniques, each of which tries to cater to different challenges.

Challenges regularization deals with:

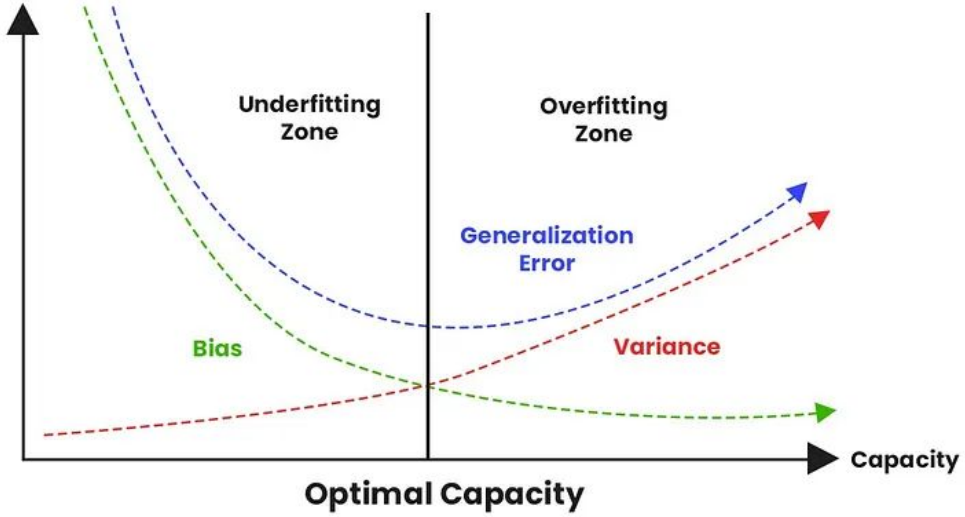
1. May put extra constraints on the learning of an ML model, like adding restrictions on the range/type of parameter values.
2. May add adds a careful selection of the right constraints and penalties in the cost function which contributes to a massive boost in the model's performance, specifically on the test dataset.
3. Extra terms can also be encoded based on some prior information that closely relates to the dataset or the problem statement.
4. Ensemble models, which take into account the collective decision of multiple models, each trained with different samples of data.

The aim of regularization is to reduce the over-complexity of the machine learning models and help the model learn a simpler function to promote generalization.


Train set
Linear regression (train acc: 0.83)
Neural network without regularisation (train acc: 0.89)
Neural network with regularisation (train acc: 0.87)
X_1
X_2

Regularization in Deep Learning

- In the context of deep learning models, most regularization strategies revolve around <u>regularizing estimators.</u>
- Bias vs variance tradeoff graph is shown below:



- Regularization of an estimator works by trading increased bias for reduced variance.
- An effective regularization will be the one that makes the best trade between bias and variance, and the end-product of the tradeoff should be a significant reduction in variance at minimum expense to bias.
- This means low variance without immensely increasing the bias value.

- Let us consider two scenarios:
  - The true data-generating process/function: F1, which created the dataset.
  - Creating a generating process/function: F2 that mimics F1 but also explores other possible generating scenarios/functions
- Regularization techniques are to help take our model from F2 to F1 without overly complicating F2.
- Since Deep learning is applied in more complicated domains like images, audio, text sequences or simulating complex decision making tasks. F1 is almost impossible to be correctly mapped, hence with regularization, it brings our model with F2 function as close as possible to the original F1 function.

- In practical deep learning, we mostly find that the best fitting model (in the sense of least generalization error) is often a large model that has been regularized appropriately.

Parameter Norm Penalties:

- Under this kind of regularization technique, the capacity of the models like neural networks, linear or logistic regression is limited by adding a parameter norm penalty Ω(θ) to the objective function J. The equation can be represented as the following:

$$\tilde{J}(\theta; X, y) = J(\theta; X, y) + \alpha\Omega(\theta)$$

where α lies within [0, ∞) is a hyperparameter that weights the relative contribution of a norm penalty term Ω, pertinent to the standard objective function J.

Setting α to 0 results in no regularization while larger values correspond to a greater regularization.

- This type of regularization penalizes only the weights of the affine transformation at each layer of the network which leaves the biases unregularized since it typically requires lesser data to fit the biases than the weights.

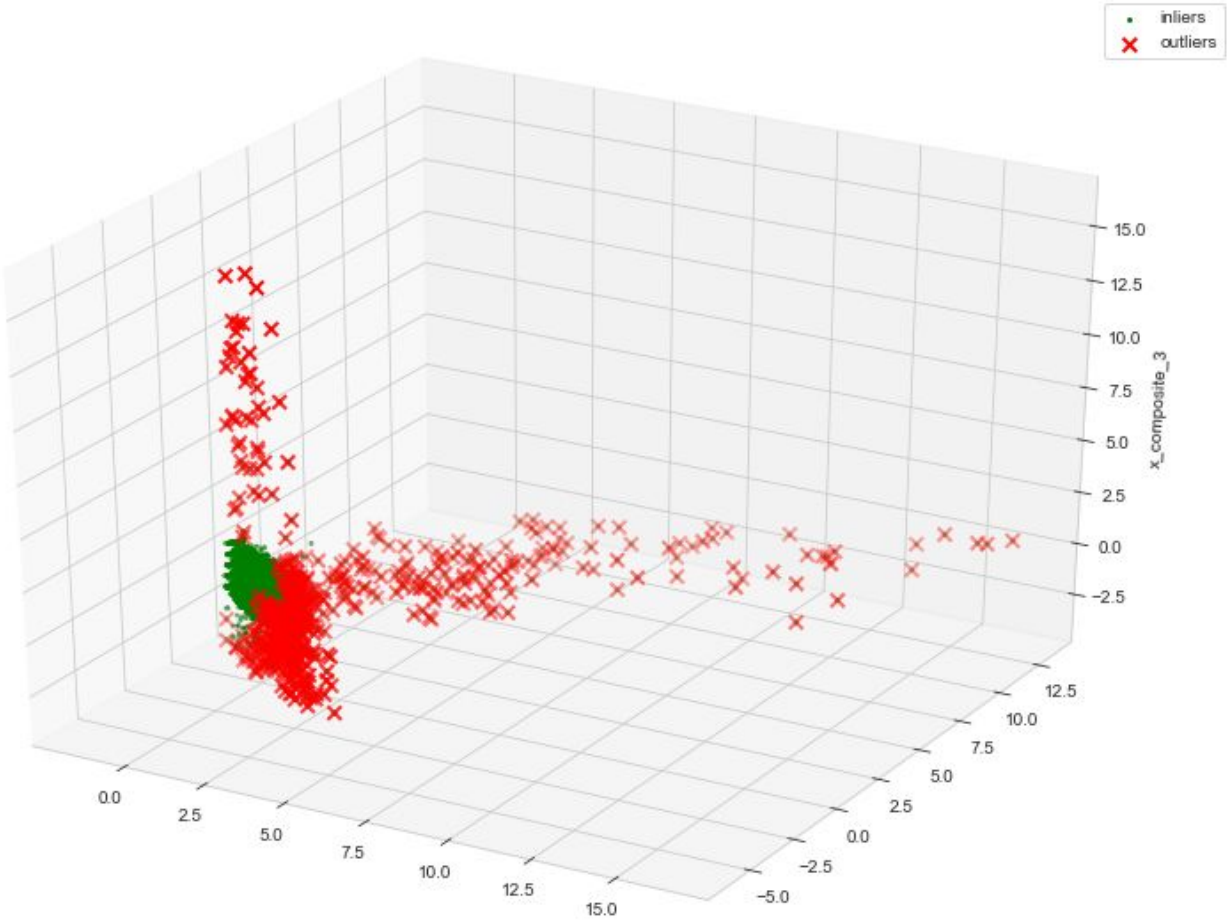
L1 Parameter Regularization:

- L1 regularization is a more specific gradient descent optimization problem.

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^{N} |\omega_i|$$

- Lasso Regression (Least Absolute Shrinkage and Selection Operator) adds “Absolute value of magnitude” of coefficient, as penalty term to the loss function.
- Lasso shrinks the less important feature’s coefficient to zero; thus, removing some feature altogether. Good for feature selection in case of huge number of features.

- The L1 regularizer looks for the parameter vectors that minimize the norm of the parameter vector (the length of the vector).
- This is essentially the problem of how to optimize the parameters of a single neuron, a single layer neural network in general, and a single layer feed-forward neural network in particular.
- L1 helps identify outliers in the distant vicinity of the dataset.


inliers
outliers
x_composite_3
15.0
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
0.0
2.5
5.0
7.5
10.0
12.5
15.0
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
12.5

- It is a method of maximizing the area of the parameter hyperspace that the true parameter vector is within.
- To do this, it finds the “sharpest” edge, one that is as close to the parameter vector as possible.

Key points

- L1 regularization is easy to implement and can be trained as a one-shot thing, meaning that once it is trained you are done with it and can just use the parameter vector and weights.
- L1 regularization is robust in dealing with outliers. It creates sparsity in the solution (most of the coefficients of the solution are zero), which means the less important features or noise terms will be zero.
- It makes L1 regularization robust to outliers.

Ex

1. To classify images of various birds of different species, we have a neural network with a few hundred parameters.
2. Find a sample of birds we are reasonably certain are of one species.
3. Add this image to the training set and try to train the neural network. This is like throwing an outlier into the mix of all the others. By looking at the edge of the hyperspace where the hyperplane is closest to, we pick up on this outlier, but by the time we've got to the hyperplane it's quite far from the plane and is hence an outlier.

4. The solution in such cases is to perform iterative dropout. L1 regularization is a one-shot solution, but in the end we are going to have to make some kind of hard decision on where to cut off the edges of the hyperspace.
5. Iterative dropout is a method of deciding exactly where to cut off. It is a little more expensive in terms of training time, but in the end it might give us an easier decision about how far the hyperspace edges are.

L2 Parameter Regularization:

- The Regression model that uses L2 regularization is called Ridge Regression.

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

$$\min_{\theta} \; J(\theta)$$

- Regularization adds the penalty as model complexity increases.
- The regularization hyperparameter (lambda) penalizes all the parameters except the bias term so that the model generalizes the data and won't overfit.

- Ridge regression adds “squared magnitude of the coefficient” as penalty term to the loss function.
- Here the second part of the equation represents the L2 regularization element/term.
- If the hyperparameter lambda is zero, then it is equivalent to OLS.
    - Ordinary Least Square or OLS, is a statistical model which helps in identifying more significant features that can have a heavy influence on the output.
- If lambda is very large, then it will add too much weight, and it will lead to under-fitting.

Key points

1. Ridge regularization forces the weights to be small but does not make them zero and does not give the sparse solution.
2. Ridge is not robust to outliers as square terms blow up the error differences of the outliers, and the regularization term tries to fix it by penalizing the weights.
3. Ridge regression performs better when all the input features influence the output, and all with weights are of roughly equal size.
4. L2 regularization can learn complex data patterns

Difference

1. In comparison to L2 regularization, L1 regularization results in a solution that is more sparse. Sparsity in this context refers to the fact that some parameters have an optimal value of zero.

2. The L1 penalty causes a subset of the weights to becomes zero, which is safe to suggest that the corresponding features associated with the respective weights, may easily be discarded. Used for feature selection

3. Many regularization methods can be understood from a Bayesian perspective. L2 regularization is equivalent to MAP Bayesian inference when the model parameters are assumed to have a Gaussian prior. Similarly, L1 regularization corresponds to MAP Bayesian inference with a Laplace (double-exponential) prior on the model parameters. (A Gaussian prior favors small but non-zero weights $\longrightarrow$ smooth solutions (L2). A Laplace prior favors many zero weights $\longrightarrow$ sparse solutions (L1))

Dataset Augmentation

- Artificially increase training data using label-preserving transformations.
- Instead of constraining weights, we expand the dataset.
  - More data $\longrightarrow$ better generalization
  - Forces the model to learn invariances
- Examples
  - Images: rotation, flipping, cropping, color jitter
  - Text: synonym replacement, back-translation
  - Audio: pitch shift, time stretch

- If original data is x~P(x), augmentation approximates:

$$\min_{\theta} \mathbb{E}_{(x,y)\sim\mathcal{D}}\mathbb{E}_{T\sim\mathcal{A}}\left[\ell\big(f_\theta(T(x)), y\big)\right]$$

- $\min_{\theta}$: Find the parameters θ that give the smallest possible loss.
- $E_{(x,y)\sim D}$: The outer expectation which is the average training loss over the dataset. Training samples are drawn from the original data distribution. Here D is the original training dataset, x input sample and y true label.
- $E_{T\sim A}$: The inner expectation which minimises average loss over all possible augmentations so that model performs well on all reasonable transformed versions of x . T is a random augmentation transformation & A is the augmentation distribution

- $f_\theta(T(x))$: Feed the augmented data into the model. T(x) is the augmented input and f_θ is the neural network model. Feed the augmented data into the model.
- $\ell(f_\theta(T(x)),y)$: Measures how wrong is the model's prediction on the augmented input is. Here ℓ is the loss function.

Dataset augmentation minimizes the expected loss over all <u>label-preserving transformations</u> of the training data, thereby improving generalization without explicitly penalizing the model parameters. It slightly increases bias while strongly decreasing variance resulting in reduced overfitting.

Noise Robustness

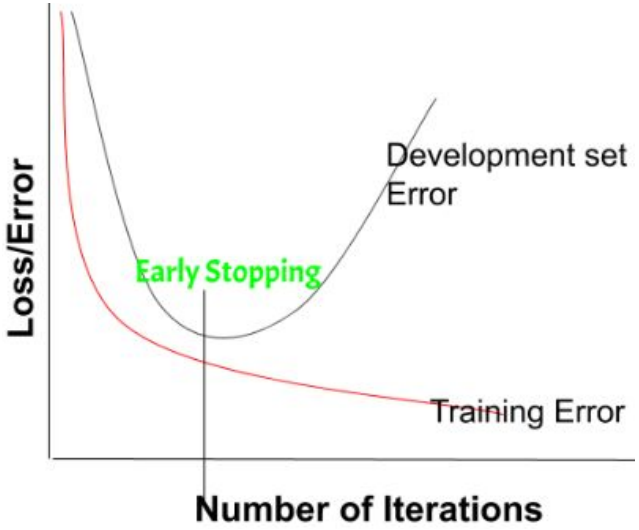
- Makes the model stable to small perturbations.
- Types of Noise
  - Input noise: $x' = x + \epsilon, \epsilon \sim N(0, \sigma^2)$
  - Label noise
  - Hidden layer noise
- Training with noise approximates: $\min_{\theta} \mathcal{L}(\theta) + \lambda \|\nabla_x \mathcal{L}\|^2$
  - L - training loss
  - $\nabla_x L$ - Gradient of loss with respect to the input. Measures how sensitive the loss is to small changes in x. High if model is sensitive to noise.
  - $\|\nabla_x L\|^2$ - Penalizes sharp decision boundaries and encourages smooth input–output mapping
  - λ - regularization strength. Controls tradeoff between accuracy & robustness

- Connection to L2
  - Input noise ≈ Tikhonov (L2) regularization under certain assumptions

Noise robustness regularization improves stability and prevents memorization of exact training samples.

Early Stopping

- During training it is observed that
  - Training loss ↓ continuously
  - Validation loss ↓ then ↑ (overfitting point)
- Solution is to stop training when validation loss starts increasing
- Prevents weights from growing too large
- Limits model capacity implicitly



- Early stopping ≈ L2 regularization in gradient descent:

$$\theta_t \approx \arg\min_{\theta} \left(\mathcal{L}(\theta) + \frac{1}{2\eta t} \|\theta\|^2 \right)$$

  - θ_t: parameters after t gradient descent steps
  - $L(\theta)$: training loss
  - η: learning rate
  - t: number of training iterations
  - $\| \theta \|^2$: L2 norm of parameters

- Early stopping implicitly limits the growth of model parameters, making it approximately equivalent to L2 regularization.
- Advantages
  - No change to loss function
  - No extra hyperparameters

Dropout

- Randomly drop neurons during training.
- Each neuron is retained with probability p.
- During training:

$$h_i^{(l)} = \begin{cases} 0, & \text{with probability } 1-p, \\ \dfrac{\phi_i^{(l)}}{p}, & \text{with probability } p \end{cases}$$

- $h_i^{(l)}$: output of neuron i in layer l after dropout
- $\phi_i^{(l)}$: activation before dropout
- p: probability of retaining (keeping) a neuron
- 1/p: scaling factor to preserve expected activation

- During training, dropout randomly deactivates neurons with probability $1-p$ and scales the remaining activations to maintain their expected value.
- During Testing:
  - Use full network
  - No dropout
- Dropout:
  - Prevents co-adaptation (neurons become dependent on each other to make correct predictions).
  - Forces neurons to be independently useful.
  - Trains an ensemble of sub-networks.

- Scaling by 1/p is done to keep the expected activation same.

$$E(h) = p.\frac{h}{p} = h$$

- Dropout causes slight increase in bias but large decrease in variance and overfitting.
- It has a similar effect to model averaging and is stronger than L2 in deep networks.

Method	Acts On	Type
Dataset Augmentation	Data	Implicit
Noise Injection	Input/Hidden	Implicit
Early Stopping	Training process	Implicit
Dropout	Network	Explicit stochastic

- An explicit stochastic regularization method deliberately introduces randomness into the training process through a clearly defined mechanism to reduce overfitting.

Method	Explicit?	Stochastic?
L2 regularization	Yes	No
Early stopping	No	No
Data augmentation	No	Yes
Dropout	Yes	Yes

- Regularization is not only about penalties on weights.It is about controlling information flow:
  - Through data (augmentation)
  - Through noise
  - Through training time
  - Through network connectivity